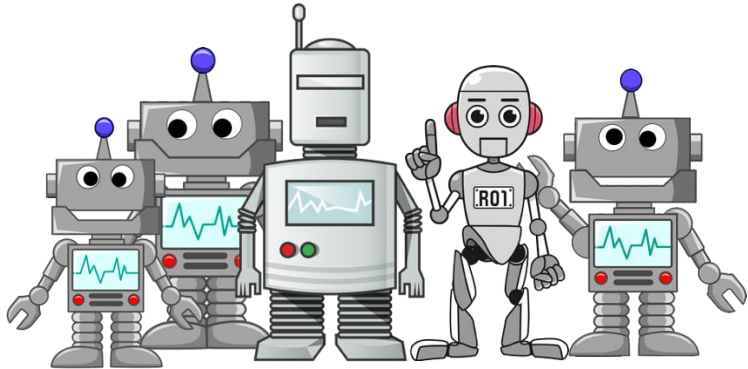


AUTONOMUS AGENTS AND MULTI-AGENT SYSTEMS (MULTI-AGENT) PLANNING

Mirgita Frasheri | mirgita.frasheri@ece.au.dk

Lukas Esterle | lukas.esterle@ece.au.dk

ROBOTS



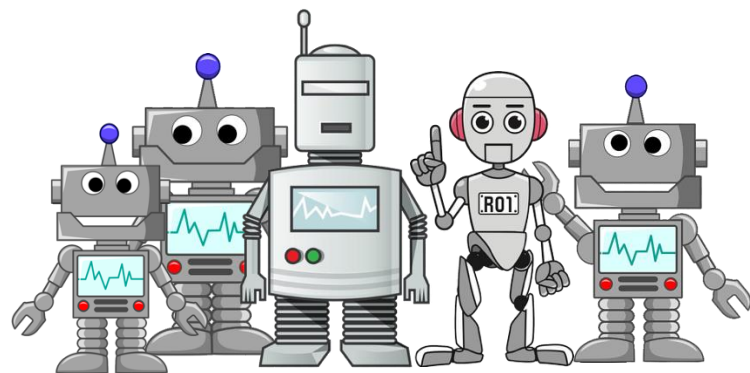
AARHUS
UNIVERSITY
DEPARTMENT OF ENGINEERING

OCTOBER 2024

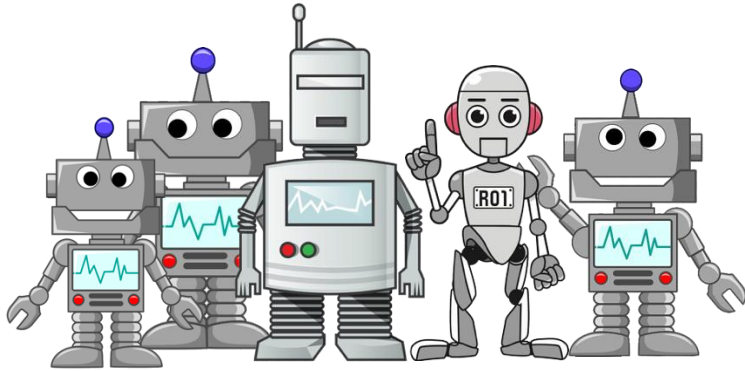
MIRGITA FRASHERI
TENURE-TRACK ASSISTANT PROFESSOR



MISSION (BUNCH OF TASKS)



MISSION (BUNCH OF TASKS)



PLANNING

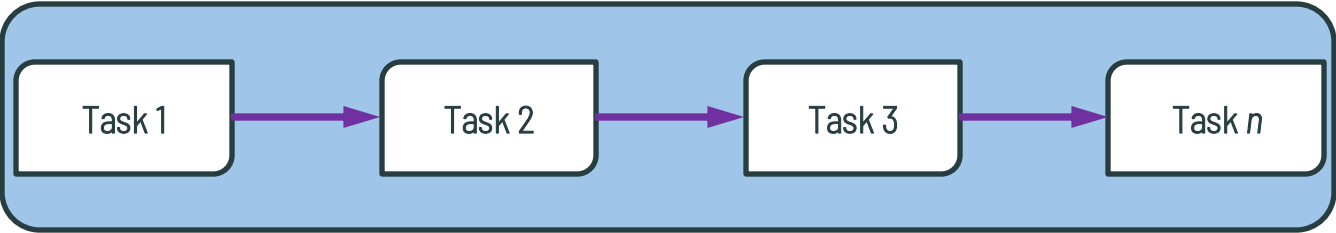
A **SEARCH** through the **SET OF TASKS**
in order to **FIND THE SEQUENCE**
that leads from the
INITIAL STATE to the **GOAL STATE**.

AN AGENT PLAN LOOKS LIKE THIS

What
&how

Plan – An ordered set of tasks

Initial State

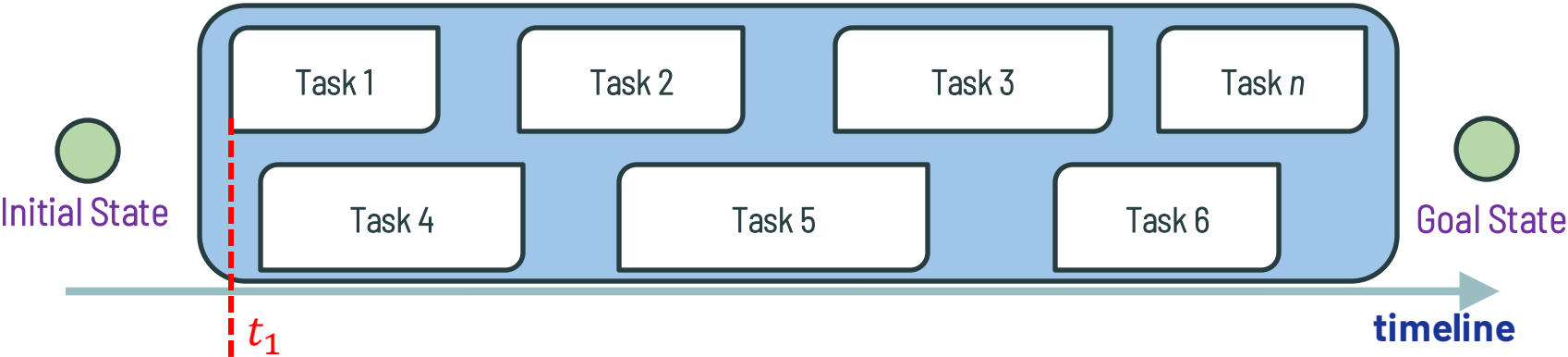


Goal State

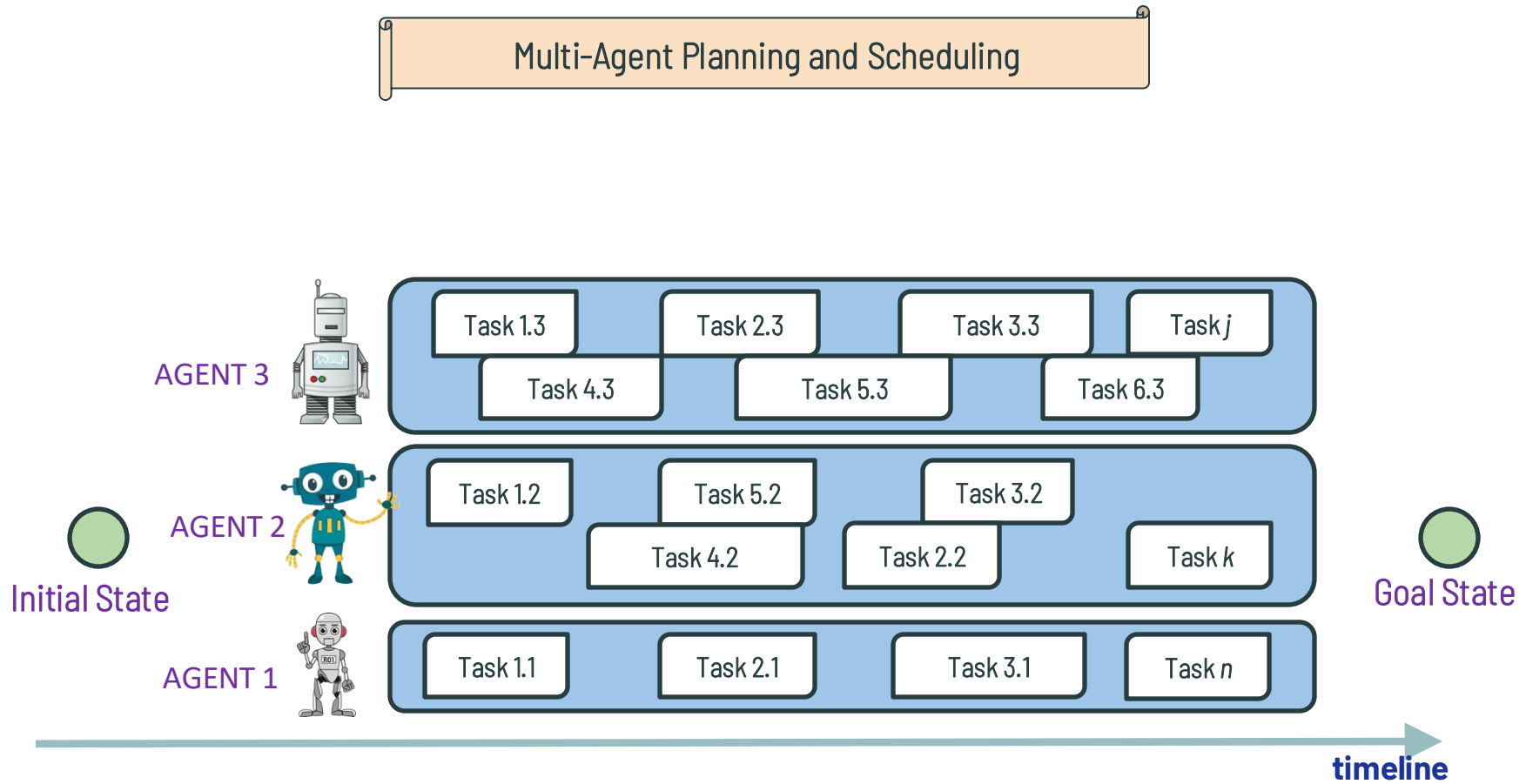
AN AGENT SCHEDULE LOOKS LIKE THIS

Who
&when

Schedule – Tasks can be performed concurrently



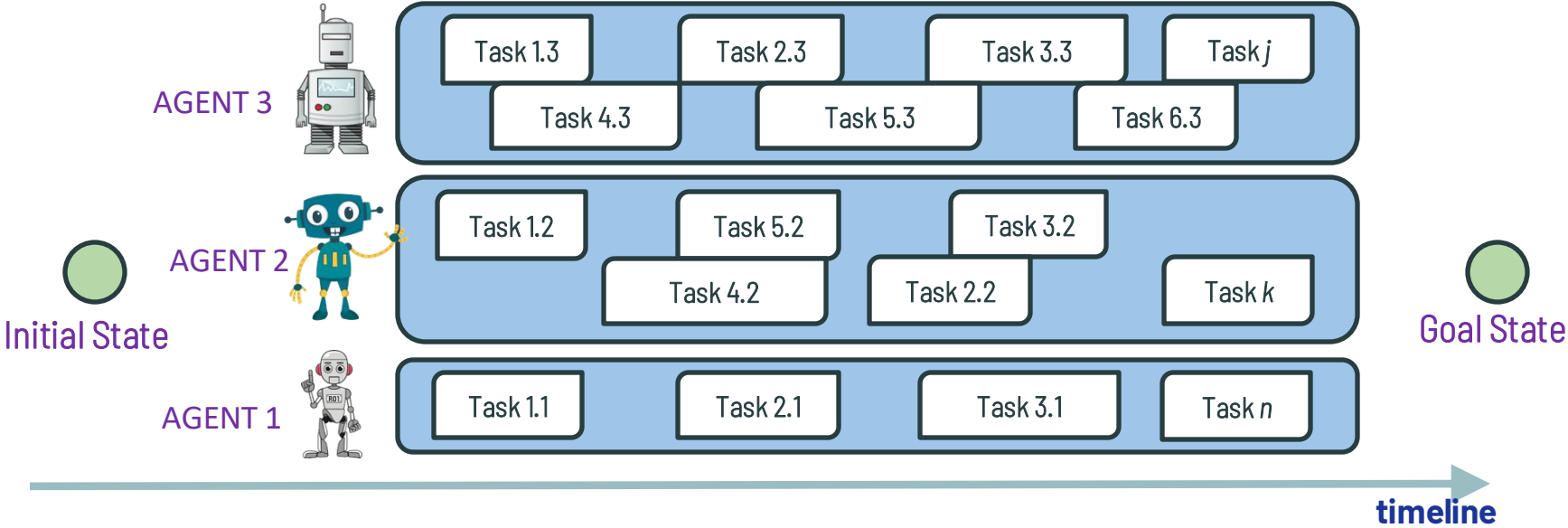
FOR A MAS WE HAVE



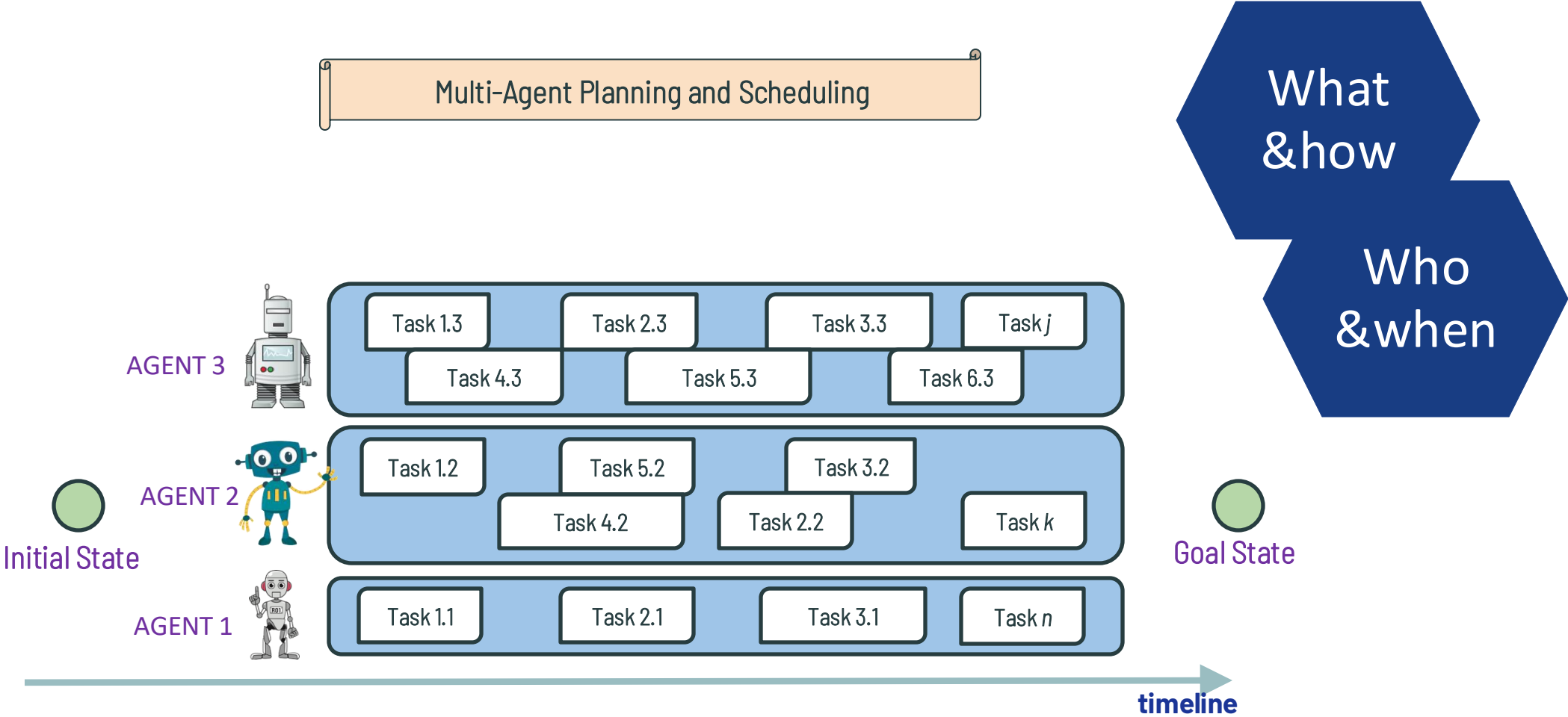
FOR A MAS WE HAVE

Multi-Agent Planning and Scheduling

What
&how



FOR A MAS WE HAVE



FOR A MAS WE ALSO HAVE:

Some additional constraints

FOR A MAS WE ALSO HAVE:

Some additional constraints

Constrained Resources (resources are not infinite).

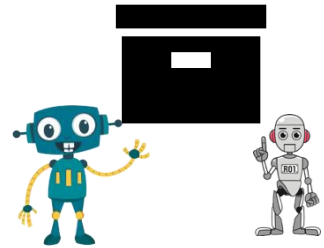


FOR A MAS WE ALSO HAVE:

Some additional constraints

Constrained Resources (resources are not infinite).

Agent Cooperation (more than one agent might be required to complete a task).



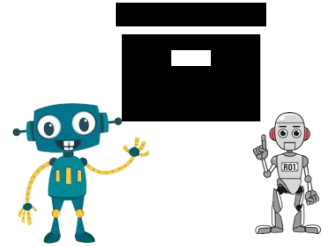
FOR A MAS WE ALSO HAVE:

Some additional constraints

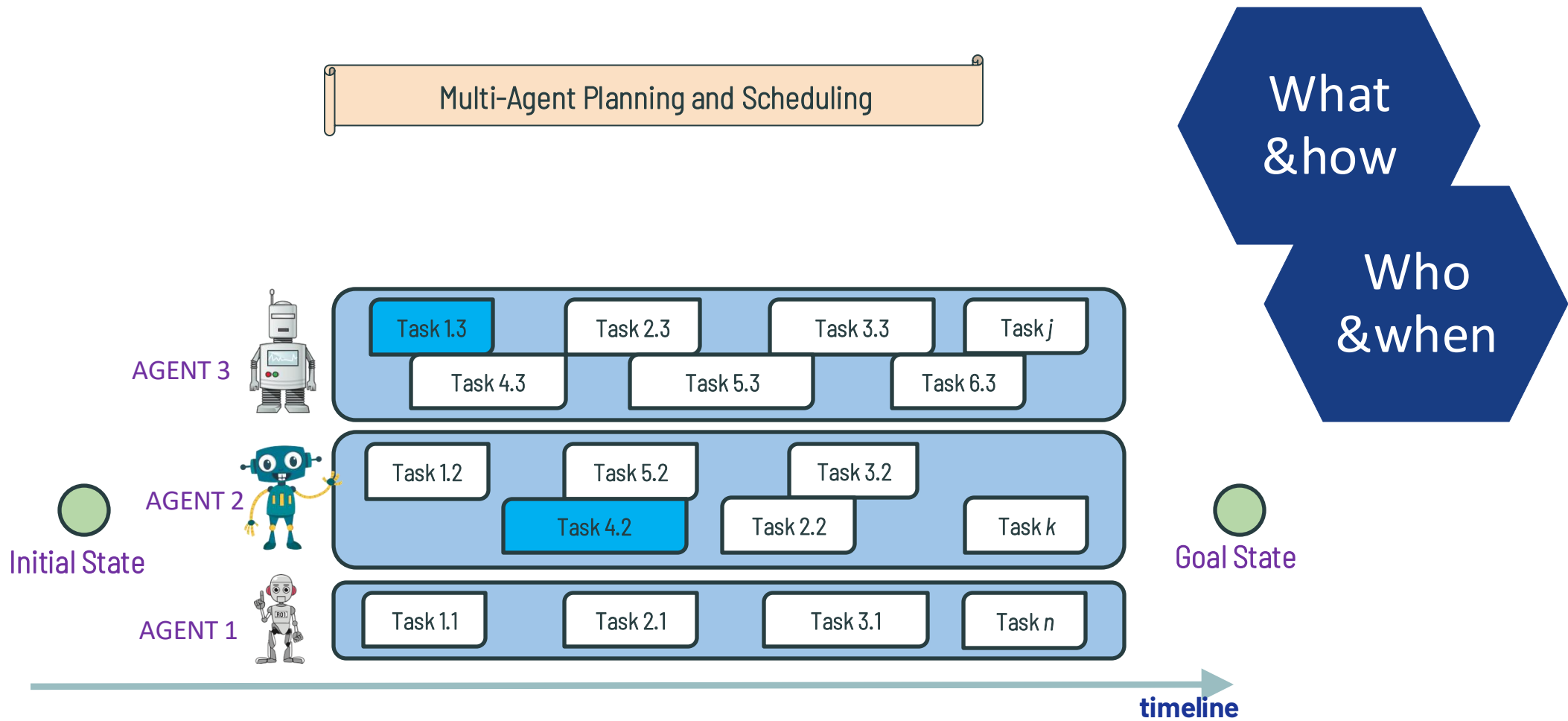
Constrained Resources (resources are not infinite).

Agent Cooperation (more than one agent might be required to complete a task).

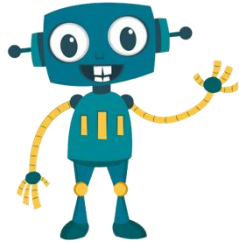
Task Dependencies completion of one task may depend on the completion of some other task



FOR A MAS WE HAVE

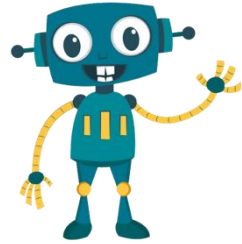


SINGLE VS MULTI-AGENT PLANNING

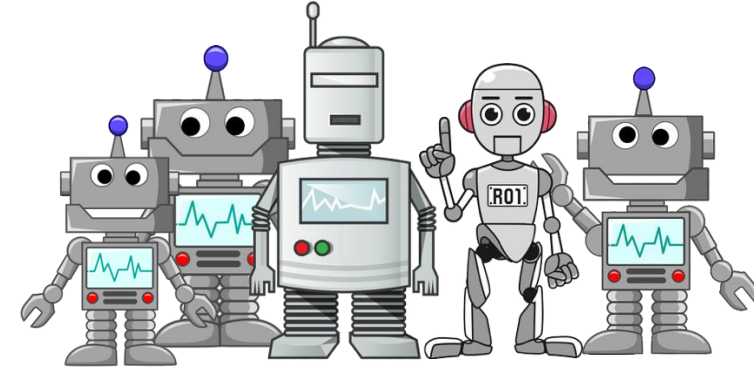


- * No Task Allocation
- * No Cross-schedule Dependencies
- * Straightforward Objective Function

SINGLE VS MULTI-AGENT PLANNING



- * No Task Allocation
- * No Cross-schedule Dependencies
- * Straightforward Objective Function

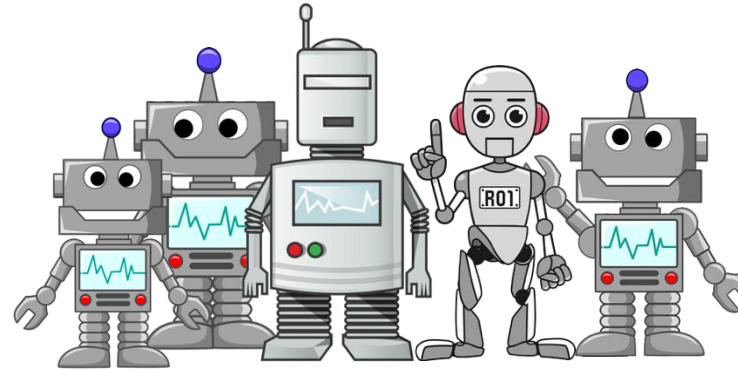


- * Task Allocation! Who Does What Now?
- * Cross-schedule Dependencies
- * More Complex Objective Function
- * Coalition Formation
- * Heterogeneous Group Structure

KEEP IN MIND

A MAS Mission can include a **large** number of:

- AGENTS
- TASKS



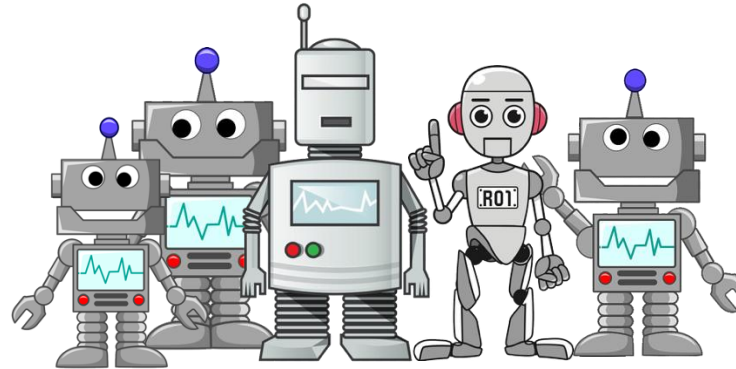
KEEP IN MIND

A MAS Mission can include a **large** number of:

- AGENTS
- TASKS

It can also contain:

- CONSTRAINTS



MAS MISSION PLANNING – WHAT IT IS NOT

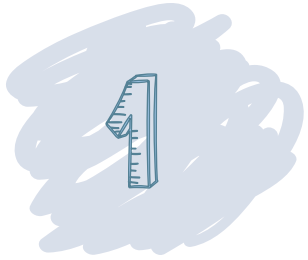
It is not path planning, task planning, motion planning, path following, path finding, etc.

MAS MISSION PLANNING – WHAT IT IS NOT

It is not path planning, task planning, motion planning, path following, path finding, etc.



TAXONOMIES FOR MULTI ROBOT TASK ALLOCATION



GERKEY AND MATARIC 2004.

Allocation type

- Instantaneous Assignment (IA)
- Time-extended Assignment (TA)

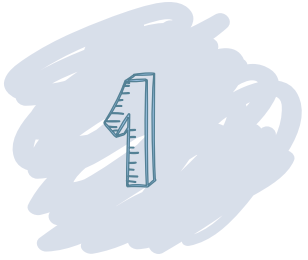
Task type

- Single Robot Tasks (SR)
- Multi-Robot Tasks (MR)

Concurrency type

- Single Task Robots (ST)
- Multi-Task Robots (MT)

TAXONOMIES FOR MULTI ROBOT TASK ALLOCATION



1

GERKEY AND MATARIC 2004.

Allocation type

- Instantaneous Assignment (IA)
- Time-extended Assignment (TA)

Task type

- Single Robot Tasks (SR)
- Multi-Robot Tasks (MR)

Concurrency type

- Single Task Robots (ST)
- Multi-Task Robots (MT)



2

KORSAH *ET AL.* 2013.

Degree of interrelatedness

- No Dependencies (ND)
- Intra-Schedule Dependencies (ID)
- Cross-Schedule Dependencies (XD)
- Complex Dependencies (CD)

TAXONOMIES FOR MULTI ROBOT TASK ALLOCATION

1

GERKEY AND MATARIC 2004.

Allocation type

- Instantaneous Assignment (IA)
- Time-extended Assignment (TA)

Task type

- Single Robot Tasks (SR)
- Multi-Robot Tasks (MR)

Concurrency type

- Single Task Robots (ST)
- Multi-Task Robots (MT)

2

KORSAH *ET AL.* 2013.

Degree of interrelatedness

- No Dependencies (ND)
- Intra-Schedule Dependencies (ID)
- Cross-Schedule Dependencies (XD)
- Complex Dependencies (CD)

TAXONOMIES FOR MULTI ROBOT TASK ALLOCATION

1

GERKEY AND MATARIC 2004.

Allocation type

- Instantaneous Assignment (IA)
- Time-extended Assignment (TA)

Task type

- Single Robot Tasks (SR)
- Multi-Robot Tasks (MR)

Concurrency type

- Single Task Robots (ST)
- Multi-Task Robots (MT)

2

KORSAH *ET AL.* 2013.

Degree of interrelatedness

- No Dependencies (ND)
- Intra-Schedule Dependencies (ID)
- Cross-Schedule Dependencies (XD)
- Complex Dependencies (CD)

3

NUNES *ET AL.* 2017.

Time-Windows (TW)

Synchronization Precedence (SP)

TAXONOMIES FOR MULTI ROBOT TASK ALLOCATION

1

GERKEY AND MATARIC 2004.

Allocation type

- Instantaneous Assignment (IA)
- Time-extended Assignment (TA)

Task type

- Single Robot Tasks (SR)
- Multi-Robot Tasks (MR)

Concurrency type

- Single Task Robots (ST)
- Multi-Task Robots (MT)

2

KORSAH *ET AL.* 2013.

Degree of interrelatedness

- No Dependencies (ND)
- Intra-Schedule Dependencies (ID)
- Cross-Schedule Dependencies (XD)
- Complex Dependencies (CD)

3

NUNES *ET AL.* 2017.

- Time-Windows (TW)
- Synchronization Precedence (SP)

THE CHALLENGE

Given a set of **heterogeneous agents**, a set of **tasks**, and a set of **constraints**, the **challenge** is how to **allocate** those tasks to agents **subject to imposed constraints**, while **minimizing** given **objective function**.

PROBLEM FORMALIZATION

Formal problem
formulation of selected
problem configurations.

HOW TO SOLVE?

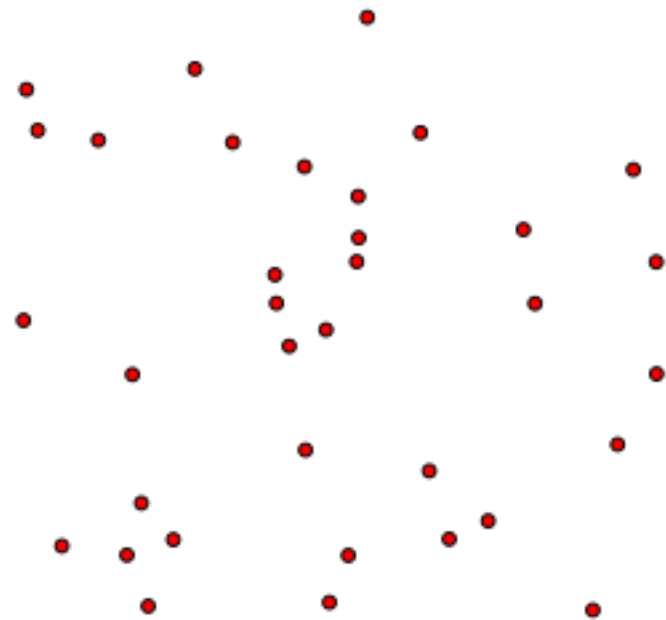
EXACT METHODS



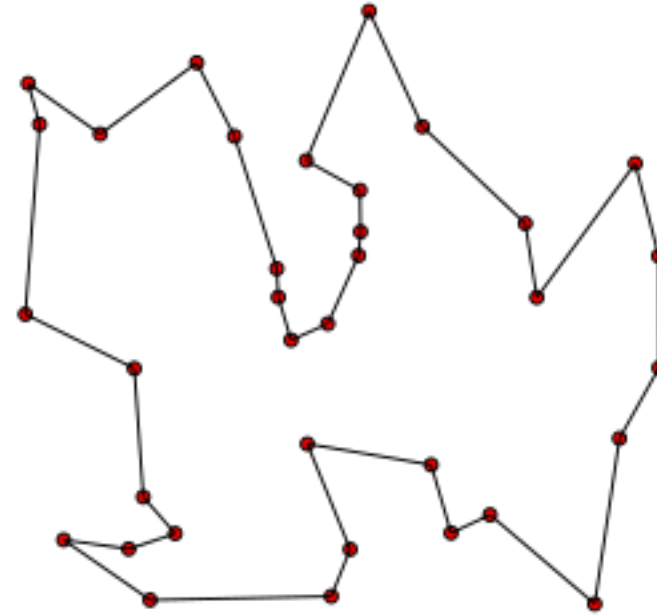
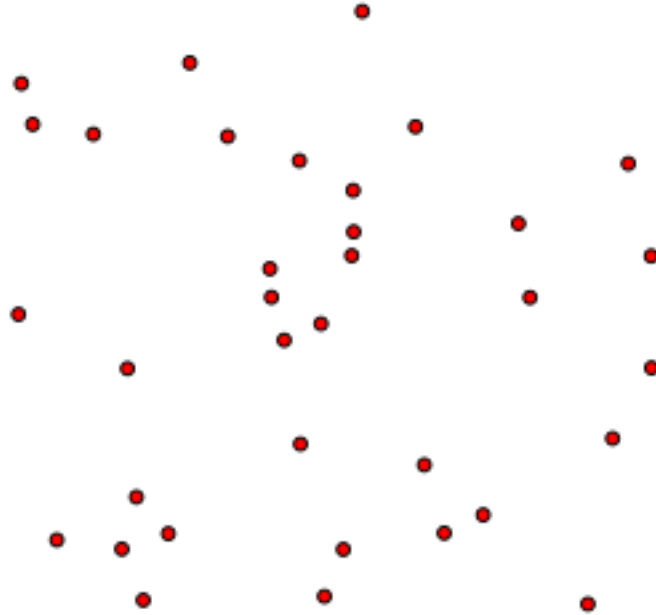
METAHEURISTICS



TSP



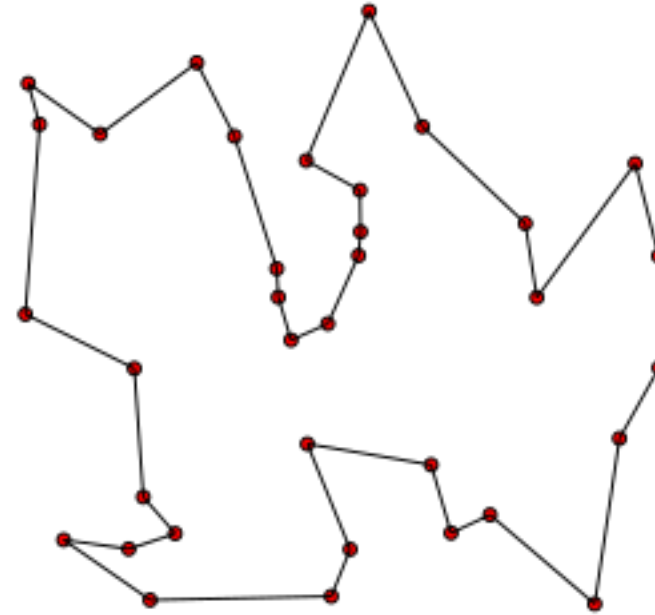
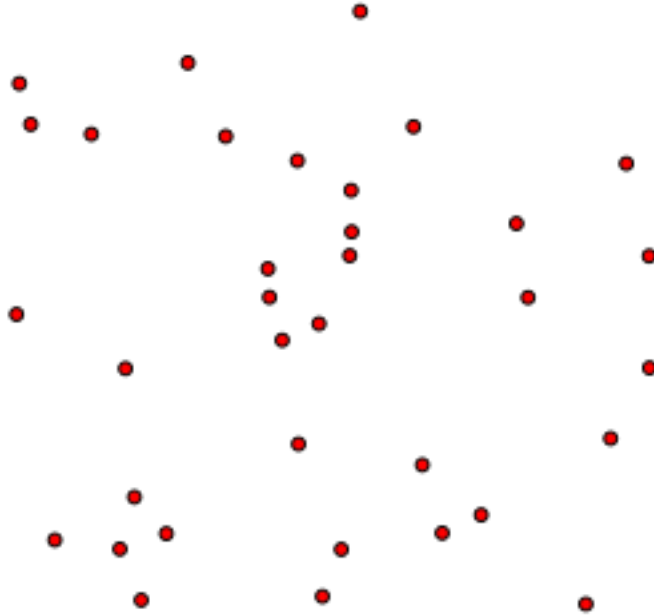
TSP



TSP

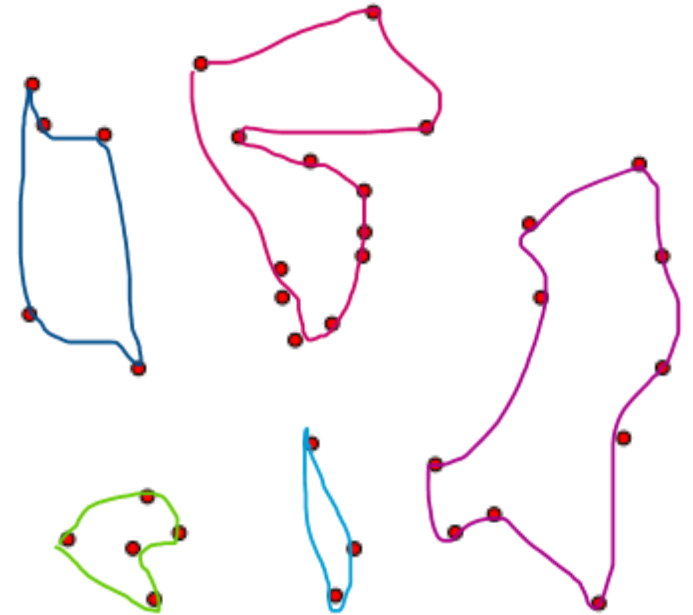
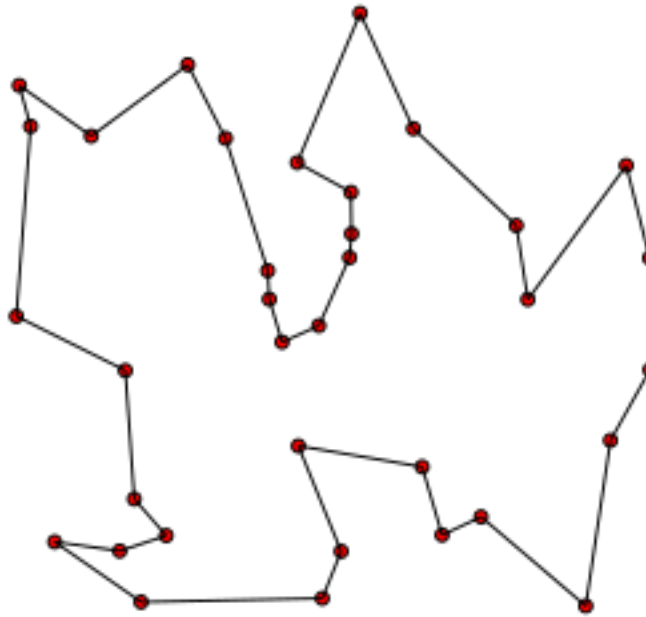
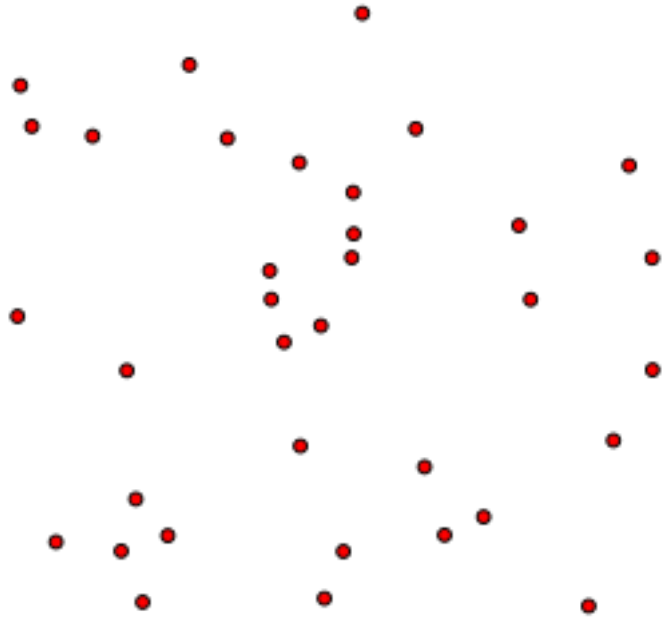
1 tour to pass through all

Objective function: e.g., min path cost



MTSP

5 salesmen/agents



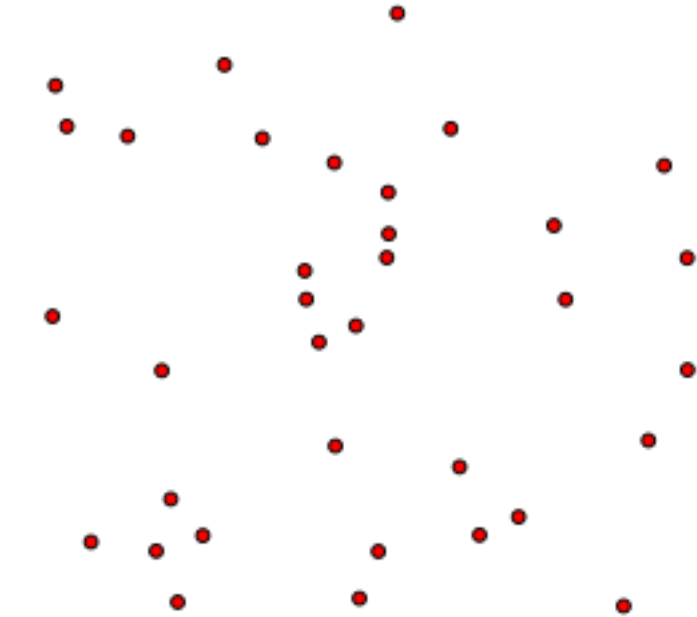
MTSP

m agents (S)

n tasks (V)

w_{ij} – cost from task i to $j \rightarrow$ stored in adjacency matrix

x_{ij} – decision variable, 1 if agent goes from i to j , 0 otherwise



MTSP – OBJECTIVE FUNCTIONS

Minimax – Minimize the interval from the start of the first task to the end of the last task

MTSP – OBJECTIVE FUNCTIONS

Minimax – Minimize the interval from the start of the first task to the end of the last task

Formally:

$$\min_x \max_s \sum_{i \in V} \sum_{j \in V} w_{ij} x_{ij}$$

MTSP – OBJECTIVE FUNCTIONS

Minimax – Minimize the interval from the start of the first task to the end of the last task

Others: minisum, combinations etc.

MTSP – CONSTRAINTS

- Only one salesperson can enter and leave a node (task)
- The starting location of a salesperson is always a source depot
- The final destination of a salesperson is always a destination depot
- The same agent enters and exits a certain city
- A salesperson cannot travel from a city to the same city
- Subtour elimination
- There can be others (later)

MTSP – CONSTRAINTS

- Only one salesperson can enter and leave a node (task)

Formally:

$$\sum_{s \in S} \sum_{i \in V} x_{ijs} = 1, \forall j \in V$$

$$\sum_{s \in S} \sum_{j \in V} x_{ijs} = 1, \forall i \in V$$

MTSP – SOLUTIONS



AARHUS
UNIVERSITY
DEPARTMENT OF ENGINEERING

OCTOBER 2024

MIRGITA FRASHERI
TENURE-TRACK ASSISTANT PROFESSOR



BRUTE FORCE

Compute all possible combinations

Pick the one satisfying your objective function (e.g., min cost)

BRUTE FORCE

Compute all possible combinations

Pick the one satisfying the objective (e.g., min cost)

Find the limit yourself

- Play with the inputs in `brute_force_mtsp.py`

SOLVING MILP FORMULATION (EXACT METHOD)

MILP → Mixed Integer Linear Programming

Linear objective function and constraints

Some or all $\mathbf{x}$ must take integer values (integrality constraints)

SOLVING MILP FORMULATION (EXACT METHOD)

Solvers: cplex, gurobi

Find the limit yourself

- Play with the inputs in `cplex_milp_mtsp.py`
- Check how the constraints are added, and what functions are used.

SOLVING MILP FORMULATION (EXACT METHOD)

Solvers: cplex, gurobi

Find the limit yourself

- The limit you see is not the actual one, due to free version of cplex
- Limit somewhere between 35-40 (you can try by using cplex' academic license)

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Individual/chromosome

```
cities= ["Gliwice", "Cairo", "Rome", "Krakow", "Paris",  
        "Alexandria", "Berlin", "Tokyo", "Hong Kong", "Rio"]
```

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Individual/chromosome

Initial population (random size and individuals)

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Individual/chromosome

Initial population (random size and individuals)

Individual = 1 possible solution → evaluate fitness

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Individual/chromosome

Initial population (random size and individuals)

Individual = 1 possible solution → evaluate fitness

In each generation → crossover, mutation on selected individuals

'random'

Best individuals →
higher selection change

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Crossover

["Gliwice", "Cairo", "Rome", "Krakow", "Paris",
"Alexandria", "Berlin", "Tokyo", "Hong Kong", "Rio"]

["Cairo", "Rome", "Krakow", "Hong Kong", "Gliwice", "Paris",
"Alexandria", "Tokyo", "Berlin", "Rio"]

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Crossover

["Gliwice", "Cairo", "Rome", "Krakow", "Paris",
"Alexandria", "Berlin", "Tokyo", "Hong Kong", "Rio"]

["Cairo", "Rome", "Krakow", "Hong Kong", "Gliwice", "Paris",
"Alexandria", "Tokyo", "Berlin", "Rio"]

["Gliwice", "Cairo", "Rome", "Hong Kong", "Gliwice", "Paris",
"Alexandria", "Tokyo", "Berlin", "Rio"]

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Crossover

["Gliwice", "Cairo", "Rome", "Krakow", "Paris",
"Alexandria", "Berlin", "Tokyo", "Hong Kong", "Rio"]

["Cairo", "Rome", "Krakow", "Hong Kong", "Gliwice", "Paris",
"Alexandria", "Tokyo", "Berlin", "Rio"]

["**Gliwice**", "Cairo", "Rome", "Hong Kong", "**Gliwice**", "Paris",
"Alexandria", "Tokyo", "Berlin", "Rio"]

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Ordered crossover

["Gliwice", "Cairo", "Rome", "Krakow", "Paris",
"Alexandria", "Berlin", "Tokyo", "Hong Kong", "Rio"]
["Cairo", "Rome", "Krakow", "Hong Kong", "Gliwice", "Paris",
"Alexandria", "Tokyo", "Berlin", "Rio"]

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Ordered crossover

["Gliwice", "Cairo", "Rome", "Krakow", "Paris",
"Alexandria", "Berlin", "Tokyo", "Hong Kong", "Rio"]
["Cairo", "Rome", "Krakow", "Hong Kong", "Gliwice", "Paris",
"Alexandria", "Tokyo", "Berlin", "Rio"]
[--, --, "Rome", "Krakow", "Paris", "Alexandria",
"Berlin", --, --, --]

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Ordered crossover

["Gliwice", "Cairo", "Rome", "Krakow", "Paris",
"Alexandria", "Berlin", "Tokyo", "Hong Kong", "Rio"]

["Cairo", "Rome", "Krakow", "Hong Kong", "Gliwice", "Paris",
"Alexandria", "Tokyo", "Berlin", "Rio"]

[--, --, "Rome", "Krakow", "Paris", "Alexandria",
"Berlin", --, --, --]

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Mutation

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Mutation

["Gliwice", "Cairo", "Rome", "Krakow", "Paris",
"Alexandria", "Berlin", "Tokyo", "Hong Kong", "Rio"]

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Mutation – Switch Step

["Gliwice", "Cairo", "Rome", "Krakow", "Paris",
"Alexandria", "Berlin", "Tokyo", "Hong Kong", "Rio"]

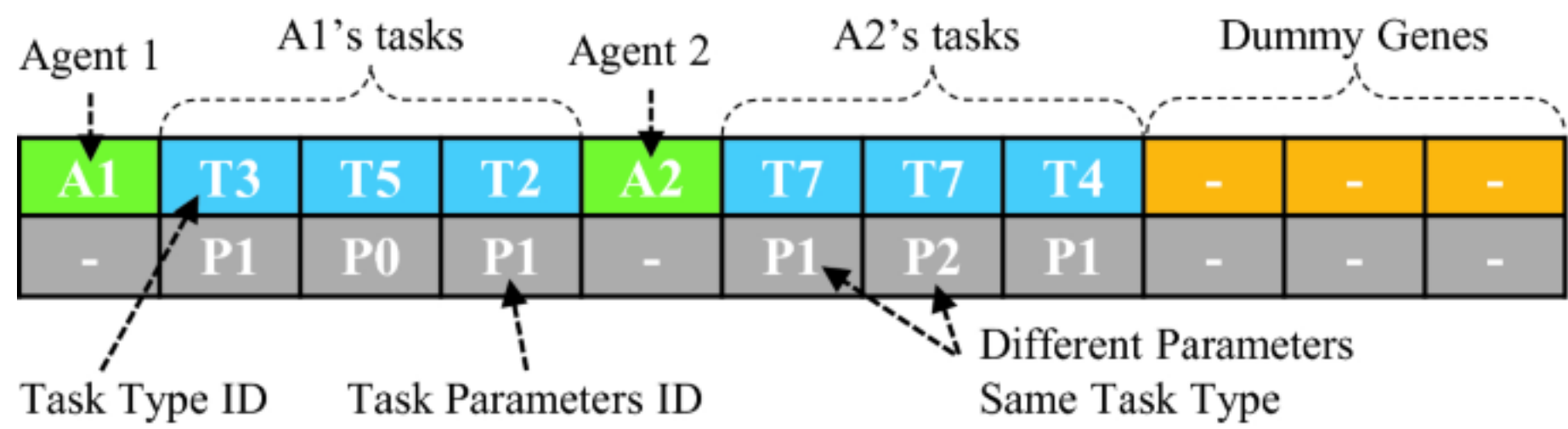
GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Mutation – Switch Step

["Gliwice", "Cairo", "Berlin", "Krakow", "Paris",
"Alexandria", "Rome", "Tokyo", "Hong Kong", "Rio"]

GA APPROACH (HEURISTIC/GOOD ENOUGH METHOD)

Multiple agents – just an example of an encoding



GA APPROACH

Find the limit yourself

- Play with the inputs in `ga_mtsp_minimax.py`

(minimax favors utilization of more agents)

MTSP – CONSTRAINTS

- Precedence constraints (task j can only be performed after task i)
- Equipment constraints (agent with camera can do ‘take image’ tasks)

YOU MIGHT NEED TO MAKE ASSUMPTIONS

* THERE ARE NO OBSTACLES IN THE ENVIRONMENT

YOU MIGHT NEED TO MAKE ASSUMPTIONS

- * THERE ARE NO OBSTACLES IN THE ENVIRONMENT
- * COMMUNICATION IS PERFECT

YOU MIGHT NEED TO MAKE ASSUMPTIONS

- * THERE ARE NO OBSTACLES IN THE ENVIRONMENT
- * COMMUNICATION IS PERFECT
- * ALL STATES ARE FULLY OBSERVABLE

YOU MIGHT NEED TO MAKE ASSUMPTIONS

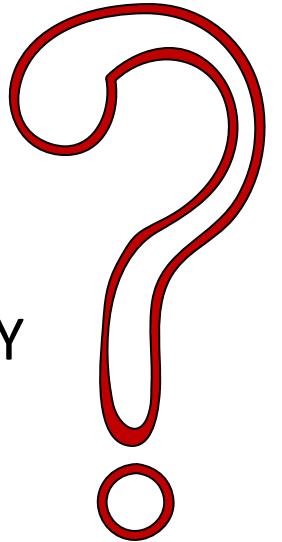
- * THERE ARE NO OBSTACLES IN THE ENVIRONMENT
- * COMMUNICATION IS PERFECT
- * ALL STATES ARE FULLY OBSERVABLE
- * PHYSICAL PROPERTIES OF ROBOTS ARE IGNORED, EXCEPT FOR VELOCITY

YOU MIGHT NEED TO MAKE ASSUMPTIONS

- * THERE ARE NO OBSTACLES IN THE ENVIRONMENT
- * COMMUNICATION IS PERFECT
- * ALL STATES ARE FULLY OBSERVABLE
- * PHYSICAL PROPERTIES OF ROBOTS ARE IGNORED, EXCEPT FOR VELOCITY
- * NO PARTIAL TASK EXECUTION IS POSSIBLE

YOU MIGHT NEED TO MAKE ASSUMPTIONS

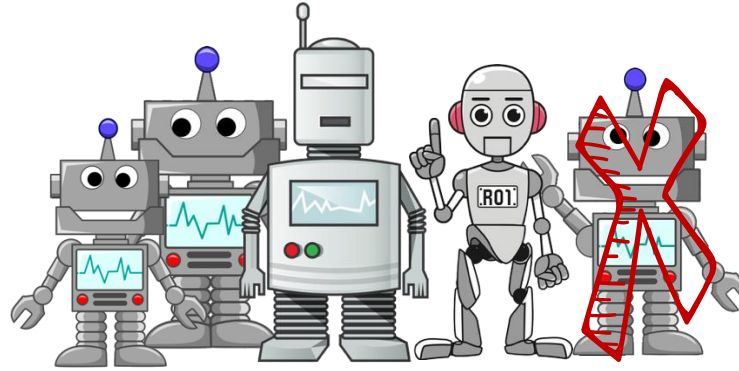
- * THERE ARE NO OBSTACLES IN THE ENVIRONMENT
- * COMMUNICATION IS PERFECT
- * ALL STATES ARE FULLY OBSERVABLE
- * PHYSICAL PROPERTIES OF ROBOTS ARE IGNORED, EXCEPT FOR VELOCITY
- * NO PARTIAL TASK EXECUTION IS POSSIBLE
- * TASK PROPERTIES ARE KNOWN A PRIORI



WHAT IF A ROBOT GOES MISSING?

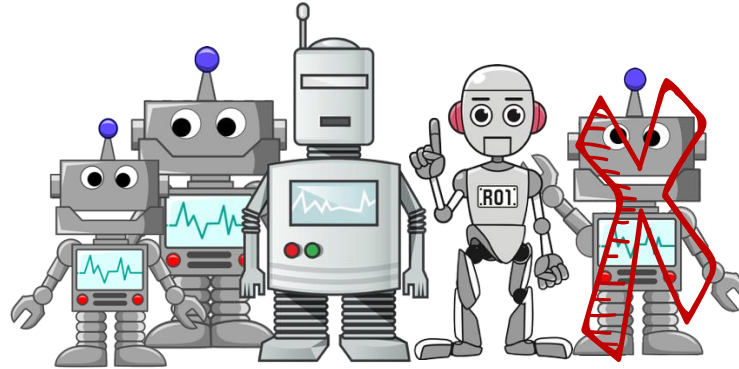
WHAT IF A ROBOT GOES MISSING?

Replan



WHAT IF A ROBOT GOES MISSING?

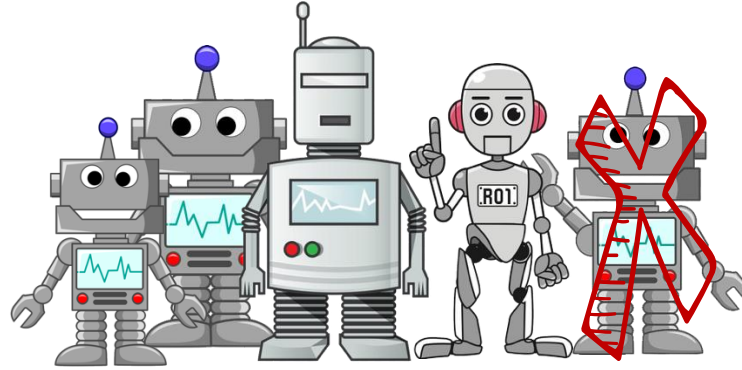
Replan



Who replans?

WHAT IF A ROBOT GOES MISSING?

Replan



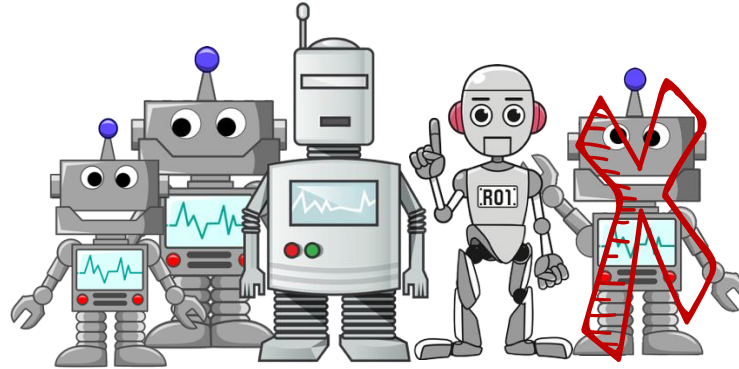
Who replans?

Global – centralised planner

Local – agents

WHAT IF A ROBOT GOES MISSING?

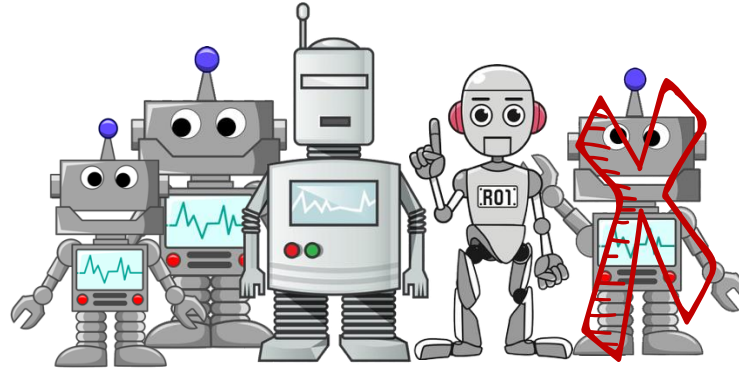
Replan



Detection of failure (full vs. partial)

WHAT IF A ROBOT GOES MISSING?

Replan

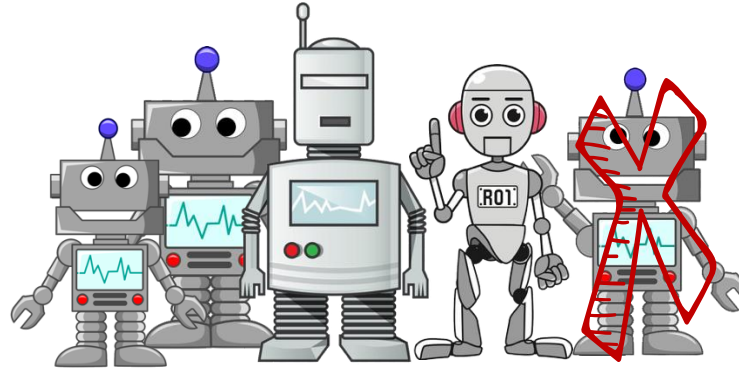


Detection of failure (full vs. partial)

Exchange of information (local vs. global view)

WHAT IF A ROBOT GOES MISSING?

Replan



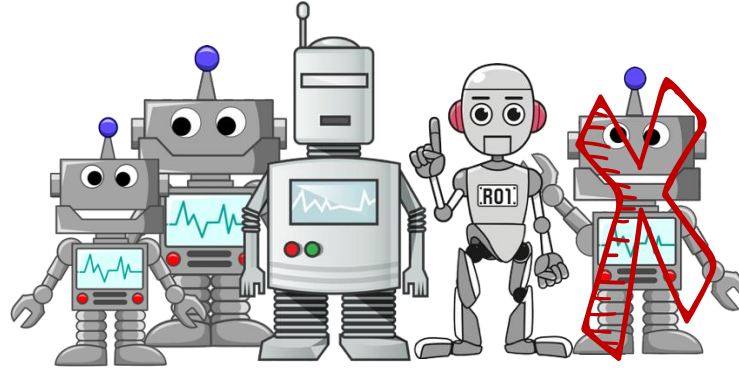
Detection of failure (full vs. partial)

Exchange of information (local vs. global view)

Handling double allocations

WHAT IF A ROBOT GOES MISSING?

Replan



Detection of failure (full vs. partial)

Exchange of information (local vs. global view)

Handling double allocations

Communication costs

WHAT IF A ROBOT GOES MISSING?

Hybrid Replanning

When should we call the planner and when should we let agents solve issues locally?

THOUGHT EXERCISE

When should we call the planner and when should we let agents solve issues locally?

Come up with different scenarios and discuss